SIMATS ENGINEERING

ASSESSMENT TOOL 2

NAME: HARITHATCHANI R

REG NO: 192421330

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0703

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

   Objective

   Calculate:

- Throughput in bits per second
- Packet loss percentage
- Network classification as Efficient or Inefficient
- Complete network performance summary

   Given Values

- Total data received = **8000 bits**
- Time = **4 seconds**
- Packets sent = **200**
- Packets received = **180**

   Formula

   Throughput:

   Packet Loss:

Since throughput is greater than 1500 bps, but packet loss is **not less than 10%**, the network is classified as **Inefficient**.

Pseudocode

```
BEGIN
  // Input values
  totalDataReceived ← 8000
  time ← 4
  packetsSent ← 200
  packetsReceived ← 180
  // Validate input
  IF time <= 0 THEN
    DISPLAY "Error: Time must be greater than zero."
    STOP
  END IF
  IF packetsSent <= 0 THEN
    DISPLAY "Error: Total packets sent must be greater than zero."
    STOP
  END IF
  IF packetsReceived < 0 OR packetsReceived > packetsSent THEN
    DISPLAY "Error: Invalid number of packets received."
    STOP
  END IF
  // Calculate throughput
  throughput ← totalDataReceived / time
  // Calculate packet loss
  lostPackets ← packetsSent - packetsReceived
  packetLossPercentage ← (lostPackets / packetsSent) × 100
  IF throughput > 1500 AND packetLossPercentage < 10 THEN
```

```
        networkStatus ← "Efficient"
    ELSE
        networkStatus ← "Inefficient"
    END IF
    // Display results
    DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
    DISPLAY "Total Data Received: ", totalDataReceived, " bits"
    DISPLAY "Time: ", time, " seconds"
    DISPLAY "Packets Sent: ", packetsSent
    DISPLAY "Packets Received: ", packetsReceived
    DISPLAY "Packets Lost: ", lostPackets
    DISPLAY "Throughput: ", throughput, " bps"
    DISPLAY "Packet Loss: ", packetLossPercentage, "%"
    DISPLAY "Network Classification: ", networkStatus
END
```

Expected Output

Total Data Received: 8000 bits

Time: 4 seconds

Packets Sent: 200

Packets Received: 180

Packets Lost: 20

Throughput: 2000 bps

Packet Loss: 10%

Network Classification: Inefficient

Why Inefficient? The throughput satisfies the requirement (>1500 bps), but packet loss must be **less than 10%**. Here it is exactly **10%**, so the condition is false.

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Objective

The algorithm continuously monitors multiple network links, stores their bandwidth utilization values, identifies overloaded links, and recommends moving traffic to a backup link when utilization exceeds **80%**.

Main Logic

1. Store all network links in an array.
2. Periodically collect utilization for every link.
3. Store the utilization values.
4. Check whether utilization is greater than 80%.
5. If overloaded, recommend a backup link.
6. Continue monitoring at regular intervals.

Pseudocode

```
BEGIN
  links ← ["Link1", "Link2", "Link3", "Link4"]
  utilization[4]
  backupLink[4]
  WHILE TRUE DO
    DISPLAY "----- BANDWIDTH MONITORING -----"
    FOR i ← 0 TO 3 DO
      utilization[i] ← GET_BANDWIDTH_UTILIZATION(links[i])
      DISPLAY links[i], " Utilization: ", utilization[i], "%"
      IF utilization[i] > 80 THEN
```

```
            DISPLAY "WARNING: ", links[i], " is overloaded."
            DISPLAY "Utilization: ", utilization[i], "%"
            DISPLAY "Recommendation: Switch traffic to ",
                backupLink[i]
        ELSE
            DISPLAY links[i], " is operating normally."
        END IF
    END FOR
    WAIT 10 seconds
  END WHILE
END
```

Example Output

```
----- BANDWIDTH MONITORING -----

Link1 Utilization: 65%
Link1 is operating normally.

Link2 Utilization: 84%
WARNING: Link2 is overloaded.
Recommendation: Switch traffic to BackupLink2

Link3 Utilization: 72%
Link3 is operating normally.

Link4 Utilization: 91%
WARNING: Link4 is overloaded.
Recommendation: Switch traffic to BackupLink4
```

How the Algorithm Supports Bandwidth Management

- **Continuous monitoring:** The WHILE loop ensures that network conditions are checked repeatedly.
- **Array storage:** Utilization values are stored for each link, making the information easy to process.
- **Overload detection:** The IF utilization > 80 condition identifies congested links.
- **Traffic redistribution:** A backup link can be used when the primary link becomes overloaded.
- **Congestion prevention:** Moving traffic away from highly utilized links helps prevent packet loss and excessive delays.
- **Scalability:** More links can easily be added to the array.
- **Real-time adaptation:** Since measurements are collected periodically, the algorithm can respond to changing network traffic.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Objective

The company has six branches connected through multiple network links. Instead of selecting a route only according to distance, the algorithm considers:

- Bandwidth
- Delay
- Utilization
- Link status

It continuously evaluates paths and selects the path with the highest overall quality score.

Link Quality Concept

A higher bandwidth is desirable, while lower delay and lower utilization are desirable.

One possible normalized score is:

Where:

BandwidthScore = CurrentBandwidth / MaximumBandwidth × 100

DelayScore = (MaximumDelay - CurrentDelay)
/ MaximumDelay × 100

UtilizationScore = 100 - CurrentUtilization

This gives more importance to bandwidth while still considering delay and utilization.

Pseudocode

```
BEGIN
  // Six branch offices
  branches ← ["Branch1", "Branch2", "Branch3",
        "Branch4", "Branch5", "Branch6"]
  // Network links
  links ← ["L1", "L2", "L3", "L4", "L5", "L6", "L7"]
  // Arrays for monitoring
  bandwidth[7]
  delay[7]
  utilization[7]
  linkStatus[7]
  qualityScore[7]
```

```
// Maximum reference values
maximumBandwidth ← 1000
maximumDelay ← 100
// Continuously monitor the network
WHILE TRUE DO
   DISPLAY "===== NETWORK PATH MONITORING ====="
   // Collect network information
   FOR i ← 0 TO 6 DO
      bandwidth[i] ← GET_BANDWIDTH(links[i])
      delay[i] ← GET_DELAY(links[i])
      utilization[i] ← GET_UTILIZATION(links[i])
      linkStatus[i] ← GET_LINK_STATUS(links[i])
      DISPLAY "Link: ", links[i]
      DISPLAY "Bandwidth: ", bandwidth[i], " Mbps"
      DISPLAY "Delay: ", delay[i], " ms"
      DISPLAY "Utilization: ", utilization[i], "%"
      DISPLAY "Status: ", linkStatus[i]
      // Check whether link has failed
      IF linkStatus[i] = "FAILED" THEN
         qualityScore[i] ← 0
         DISPLAY "ALERT: ", links[i], " has failed."
      ELSE
         // Check for overloaded link
         IF utilization[i] > 80 THEN
            DISPLAY "WARNING: ", links[i],
                  " is overloaded."
         END IF
```

```
        // Calculate normalized scores
        bandwidthScore ←
            (bandwidth[i] / maximumBandwidth) × 100
        delayScore ←
            ((maximumDelay - delay[i])
            / maximumDelay) × 100
        utilizationScore ←
            100 - utilization[i]
        // Calculate overall link quality
        qualityScore[i] ←
            (0.5 × bandwidthScore) +
            (0.3 × delayScore) +
            (0.2 × utilizationScore)
    END IF
END FOR
// Select best available path
highestScore ← -1
bestPath ← "No available path"
FOR i ← 0 TO 6 DO
    IF linkStatus[i] = "ACTIVE" AND
       utilization[i] <= 80 THEN
        IF qualityScore[i] > highestScore THEN
            highestScore ← qualityScore[i]
            bestPath ← links[i]
        END IF
    END IF
END FOR
```

```
        // Display selected path
        DISPLAY "--------------------------------"
        DISPLAY "Best Communication Path: ", bestPath
        DISPLAY "Path Quality Score: ", highestScore
        // Handle failure or overload
        IF bestPath = "No available path" THEN
            DISPLAY "ALERT: No suitable primary path available."
            DISPLAY "Search for alternate path."
            alternatePath ← FIND_ALTERNATE_PATH()
            IF alternatePath EXISTS THEN
                DISPLAY "Alternate Path Selected: ",
                        alternatePath
                DISPLAY "Administrator Alert Generated."
            ELSE
                DISPLAY "CRITICAL ALERT: No alternate path available."
            END IF
        ELSE
            DISPLAY "Traffic routed through: ", bestPath
        END IF

        // Wait before the next monitoring cycle
        WAIT 10 seconds
    END WHILE
END
```

Example

Suppose three possible paths have the following conditions:

Path	Bandwidth	Delay	Utilization	Status
Path A	800 Mbps	30 ms	60%	Active
Path B	600 Mbps	15 ms	40%	Active
Path C	900 Mbps	70 ms	90%	Active

Although Path C has the highest bandwidth, it has 90% utilization, making it overloaded. Therefore, the algorithm can reject it and compare the quality scores of Paths A and B.

This is better than simply selecting the shortest or highest-bandwidth path because it considers the current condition of the network.

Advantages of the Algorithm

1. Improved Reliability

The algorithm continuously checks whether links are active or failed. If a link fails, an alternate route can be selected.

2. Better Routing Efficiency

Instead of using only distance, the algorithm considers bandwidth, delay, and utilization to determine the most suitable route.

3. Fault Tolerance

When a primary link fails or becomes unsuitable, the system searches for an alternate path and generates an administrator alert.

4. Congestion Prevention

Links with utilization above **80%** are identified as overloaded. Traffic can be redirected before severe congestion occurs.

5. Dynamic Adaptation

Because the measurements are collected repeatedly, the selected path can change as network conditions change throughout the day.

6. Continuous Monitoring

The WHILE loop provides continuous monitoring, while the FOR loop checks every network link during each monitoring cycle.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient